

دليل المدرب الشامل وإدارة الورشة



دليل المدرب

Trainer Playbook · Delivery, Facilitation & Assessment

المخرجات / Evidence 6 أوراق + بطاقات مدموجة + موقع منشور + إصدار v1.0.0	عدد الطلاب الأمثل / Optimal class 20-12 طالباً	الأدوات / Tools Git + GitHub + VS Code + المتصفح	المشروع / Project class-team-hub (موقع ثابت بلا تبعيات)	الفئة / Audience متدربون بالغون — مستوى متوسط	مدة البرنامج / Programme 5 جلسات · 7.5 ساعة عمل موجهة (+ وحدة متقدمة 100 د)
---	--	--	--	---	--

دليل المدرب / Trainer guide

1 · فلسفة البرنامج ونتائجه Programme philosophy & outcomes

البرنامج مبني على مبدأ واحد: **يُتعلَّم Git بالعمل الجماعي فقط**. لذلك يبدأ كل طالب بعمل خاص به (بطاقته)، ثم يرى عمل زملائه، ثم يواجه تعارضاً حقيقياً ويحلّه، ثم يُنشر النتيجة للعالم. في نهاية البرنامج يكون المتدرب قادراً على العمل كعضو فريق في مستودع حقيقي — لا أن يحفظ أوامر فقط.

EN

The programme rests on one principle: Git is learned through real collaboration. Each trainee owns a personal artefact (their card), sees teammates' work, resolves a genuine conflict, and ships the result publicly.

النتائج التدريبية المتوقعة (Measurable outcomes):

# النتيجة	كيف تُقاس
1 إنشاء حساب ومستودع وإعداد بيئة عمل مؤمنة	ورقة 01 — الأدلة السبعة
2 استنساخ مشروع حقيقي وفهم بنيته وتشغيله محلياً	ورقة 02 — دليل التشغيل + جدول البنية
3 مساهمة كاملة: فرع → التزام → رفع → طلب سحب → مراجعة ورقة 03 — رقم PR + مراجعة مكتوبة	
4 مزامنة عمل الفريق وفهم استراتيجيات الدمج	ورقة 04 — مخرجات الدمج + تقرير الفريق
5 حل التعارضات بأمان والنشر على الإنترنت	ورقة 05 — دليل تعارضين + رابط منشور

مبدأ التصميم التربوي Instructional design principle

كل ورقة = دورة تعلّم كاملة: مفهوم قصير → تنفيذ على الجهاز → دليل مخرجات → تحقق ذاتي → تسليم. لا يوجد بند نظري بلا تنفيذ مقابله، ولا تنفيذ بلا دليل يثبته. هذا يقلّل «الحفظ الأعمى» ويمنحك أنت أدلة قابلة للمطابقة.

2 · تخطيط الجلسات (خطة تنفيذ مقترحة) Session plan

الجلسة الورقة	الزمن محتوى موجه	مخرجات الجلسة	تقنية	عمل منزلي
1 01 التهيئة	90 د مفاهيم Git/GitHub · المناطق الثلاث · تثبيت وضبط ودعم 2FA	حساب مؤمّن Git + مهياً	عرض s1-intro 20 د + تنفيذ 60 د + تحقق 10 د	إتمام المهام 1-4

الجلسة الورقة	الزمن محتوى موجّه	مخرجات الجلسة	تقنية	عمل منزلي
2	02 الاستنساخ 75 د	clone · بنية المشروع · تشغيل محلي	الموقع يعمل على الأجهزة + Issue	عرض s2-clone 10 د + تنفيذ 55 د قراءة docs/git-cheatsheet.md
3	03 الفرع وطلب السحب 90 د	الفرع · الالتزام المهني · PR · المراجعة	بطاقة كل طالب + PR + مراجعة زميل	عرض s3-branch-pr 10 د + تنفيذ موجّه 60 د + مراجعة ثنائية 20 د استجابة لملاحظات المراجع
4	04 المزامنة والدمج 90 د	fetch/pull · الاستراتيجيات · التراجع	main محدّث + تقرير الفريق	عرض s4-merge 10 د + تنفيذ 55 د + مناقشة تاريخ المشروع 25 د قراءة sandbox/conflict-lab.md
5	05 التعارضات والنشر 120 د	التعارض · rebase · النشر · التسليم	تعارضان + حلولان + موقع منشور + تقرير	عرض s5-conflicts 15 د + مختبر تعارض 45 د + نشر 30 د + احتفال 30 د مهمة إثراء اختيارية
6	06 الأتمتة (وحدة متقدمة اختيارية) 100 د	CI · GitHub Actions · حماية الفرع · CODEOWNERS · الإصدارات	فحص أخضر + إصدار v1.0.0 + حماية فرع	عرض s6-automation 20 د + فحص مباشر 30 د + تجربة كسر/إصلاح 30 د + إصدار 20 د فحص جديد لخطأ متكرر

الوحدة المتقدمة (السادسة) — لمن ومتى؟ The advanced unit

الورقة 06 ليست إلزامية للجميع. اعرضها على: 1. من أنهى الأوراق الخمس بنتيجة +85 (تحدّ مناسب بلا إحباط). 2. الطلاب الذين يظهرونهم بشكل متكرر — بمعنى آخر: من يصبح **مصدر خطأ متكرر** للفريق، إذ تحلّ الآلة محلّ إرشادك المتكرر. والقاعدة التربوية: امنح هذه الوحدة كمسؤولية («أنت المسؤول عن جودة المشروع الآن») لا كمكافأة، فتتحول من «واجب إضافي» إلى **منصب** يريدونه.

إدارة الوقت — أشد مشكلة في هذه الورشة Time management

90% من التأخير سببه طالب واحد عالق في تثبيت أو أمر خاطئ. الحلول المجربة: 1. **زميل مساعد**: عيّن الطالب الأسرع «مساعد الجلسة» يتحرك بين الصفوف (وليست مهمة إدارية بل تدريبية). 2. **مهل زمنية مرئية**: اكتب على السبورة وقت نهاية كل مهمة، وناذ عند الانتقال. 3. **لا تحلّ جهاز طالب بيدك**: اسأل «ماذا تقرأ في هذه الرسالة؟» واتركه ينقذ. 4. **شريحة زمن احتياطية**: اترك آخر 10 دقائق بلا محتوى جديد دائماً.

قبل اليوم الأول بثلاثة أيام / Three days before

- ☐ [] ارفع نسخة المشروع `class-team-hub` إلى منظمة GitHub خاصة بالورشة (مثال: `school-git-workshop`)
- ☐ [] افعّل حماية الفرع: `Settings ← Branches ← main ← Require a pull request before merging`
- ☐ [] ارفع إلى المنظمة ملفات القالب: `CONTRIBUTING.md` , `CODE_OF_CONDUCT.md` , `.github/PULL_REQUEST_TEMPLATE.md`
- ☐ [] جهّز تفعيل GitHub Pages (بعد الجلسة الخامسة) واختبر الرابط بمشروع تجريبي
- ☐ [] اختبر الحساب والدعوات على جهاز واحد للتحقق من أن المستودع عام وقابل للاستنساخ
- ☐ [] اطبع أوراق العمل الخمس (وكل ورقة على وجهين) + انسخ نماذج الحلول لنفسك
- ☐ [] افتح عرض الجلسة من `slides/` على شاشة العرض — ثلاث صيغ: `*.pptx` (تعديل + ملاحظات محاضر في Presenter View) و `html/index.html` (متصفح) و `pdf/*.pdf` (عرض بلا تعديل) — توزّع رقمياً لا تُطبع
- ☐ [] جهّز بطاقات إجابة/تصحيح إلكترونياً أو ورقياً حسب بيئة المؤسسة
- ☐ [] شغّل `python3 tools/verify_package.py` — يجب أن يخرج «✓ الحزمة سليمة» قبل الطباعة والرفع

قبل الجلسة بعشر دقائق / Ten minutes before each session

- ☐ [] جهاز المدرّب موصول بالشاشة، والطرفية بخط كبير وواضح
- ☐ [] المتصفح مفتوح على صفحة المستودع + تبويب Pull requests
- ☐ [] ملاحظة على السبورة: «لا عمل على main» + رابط المستودع + رابط المرافق
- ☐ [] الأوراق المطبوعة لكل الطلاب على الطاولة
- ☐ [] فحص سريع للشبكة: `git ls-remote <url>` من جهازك

محاكاة بدون إنترنت (سكرتير الإنقاذ) · Offline contingency plan

إذا سقط الإنترنت (شائع في القاعات)، لا تُلغِ الجلسة. استخدم مستودعاً محلياً وهمياً:

```
# على جهاز المدرّب / instructor machine
$ mkdir -p /srv/remote.git && git init --bare /srv/remote.git
$ git remote add origin /srv/remote.git      # بدل رابط GitHub
$ git push -u origin main
```

تربط أجهزة الطلاب بالمجلد المشترك داخل الشبكة المحلية، فتكمل تدريبات الفروع والدمج والتعارضات كاملة بلا أي وصول للإنترنت، ثم تُزامن الرفع إلى GitHub لاحقاً عندما يعود. هذا أيضاً هو السكرتير الذي وُلد منه هذا البرنامج، وقد استُخدم في توليد نماذج الحلول.

4 · إدارة الصف والمراجعة · Classroom & review management 4

أزواج المراجعة (Review pairings)

لا تدع الطلاب يختارون مراجعيهم بحرية تامة؛ الأصدقاء يتبادلون المدح. استخدم دوران (rotation): صفّ الطلاب، وكل طالب يراجع من يجلس بعيداً عنه. غير الاقتران في كل ورقة.

EN

Rotate reviewers deliberately. Friends swap praise; rotation produces real feedback.

الجلسة نمط الاقتران	عدد المراجعين لكل طلب سحب هدف المهارة	
ثنائي (2 × 2)	1	كتابة أول مراجعة مكتوبة
ثلاثي حسب الأدوار (مطوّر + مختبر + كاتب)	2-1	مراجعة متخصصة
مراجعة مفتوحة (كل طالب على طالبين)	2	نقد تواريخ الدمج والنشر

سَلَم تقييم جودة المراجعة (علّمه للطلاب قبل التطبيق)

المستوى الوصف	مثال جيد من مشروعنا
5 نجوم احترام	ملاحظة محددة + سؤال + اقتراح تعديل + «سطر 8: fullName اسم مختصر جداً، هل نضيف اسمك الكامل ليظهر صحيحاً في البطاقة؟»
3 نجوم	ملاحظة صحيحة بلا اقتراح «حقل tags فارغ»
نجمة	موافقة بلا تفصيل «ممتاز»
صفر	نقد شخصي أو سخرية أو صمت —

قواعد التقدير التي أنصحك بإعلانها مسبقاً *Announce these marking rules up front*

- الأدلة هي الحكم: لا تحتسب مهمة بلا مخرجات أمر حقيقية من جهاز الطالب.
- المراجعة نصف الدرجة في الورقة 3، وربعها في 4 و5. من لا يراجع لا يُقَيَّم كامل الدرجة.
- الدقة التقنية يُسامح عليها، والمخالفات الأمنية لا: نسيان أمر = مثال تعليمي؛ رفع token أو --force على فرع مشترك = خصم مباشر.
- الفهم العملي مقدّم على الجمالية: ملف بـ 4 مسافات في الأخطاء البصرية لا يُخصم عليه؛ ملف بلا التحقق (node tests/validate-members.mjs) يُخصم عليه.
- حق إعادة المحاولة: لكل طالب إعادة تسليم واحدة خلال 48 ساعة على الأجزاء التي فشلت فقط، بشرط أن يرفق مخرجات جديدة.

5 · الأخطاء الشائعة وخط التدخل 5 · Common failures & interventions

#	المشكلة	السبب الجذري	التدخل الفوري (2 دقيقة)	التدخل العميق
1	fatal: not a git repository	نُفذ الأمر خارج مجلد المشروع	pwd ثم	علّمه علامة الفرع في الموجه
2	Author identity unknown	لم يضبط user.name/email	git config --global ... مباشرة	ربط بريده بحساب GitHub
3	non-fast-forward	لم يُحدّث الفرع	git pull --rebase ثم push	درس fetch في الورقة 04
4	بطاقة لا تظهر في الصفحة	نسي build-index.mjs	node tools/build-index.mjs	شرح الملف المؤلّد
5	<<<<<<< في الملف المرفوع حل تعارض ناقص		grep -rn "<<<<<<<" ثم الإصلاح	مختبر الورقة 05
6	JSON غير صالح (فاصلة زائدة) كتابة يدوية بلا محرر JSON		node tests/validate-members.mjs	استخدام VS Code مع تلوين JSON
7	الطالب يدمج فرعه بنفسه تجاوز عملية المراجعة		أوقف الدمج واطلب مراجعاً في الحال	خصم موثّق + مناقشة

# المشكلة	السبب الجذري	التدخل الفوري (2 دقيقة)	التدخل العميق
8 عمل مشترك على جهاز واحد	قلة الأجهزة	توزيع الأدوار: عضو تنفيذي + عضو موجه	تخطيط بديل بمستودع مشترك محلي
9 «ضاع عملي»	<code>reset --hard</code> أو <code>restore</code>	<code>git reflog</code> ثم <code>git switch -c rescue <sha></code>	درس التراجع في الورقة 04
10 تعارض متكرر في نفس الملف	عدلاً نفس السطر في نفس الملف	أخبرهما بقاعدة تقسيم الملفات	إعادة توزيع المهام
11 «الفحص الأحمر أزعجني» فرغت بالقوة»	لم يفهم أن الحماية تحمي الفريق	أوقف الدفع، واعرّض سبب الفشل على الشاشة	تدريبات الورقة 06 (كسر/إصلاح)
12 <code>validate-members.mjs</code> ليُخفي خطأه	سلوك غير مهني (تجاوز الفحص)	راجع تعديله وأعد الحالة	مناقشة فردية + خصم موثّق
13 الفحص يعمل عند الطالب ولا يعمل في CI	إصدار Node مختلف أو اعتماد على ملف غير متتبع	تُبّت إصدار Node في <code>setup-node</code> وتأكّد من <code>git status</code> نظيف	درس: بيئة CI معزولة ونظيفة

قاعدة الدقيقتين The two-minute rule

أي مشكلة لم تُحلّ مع الطالب خلال دقيقتين: سجّلها على السبورة في «طابور المشكلات»، وانتقل لغيره، ثم عالجها جماعياً في نهاية الجلسة. الطالب الذي يشغل المدرّب 15 دقيقة يحرم 15 طالباً آخرين من التوجيه — وهذا نمط شائع في ورش البرمجيات.

6 . أسئلة متوقعة من الطلاب (وأجوبتك النموذجية) Expected questions · 6

السؤال	الجواب المختصر الذي تعطيه
هل يمكن الرجوع بعد <code>commit</code> خاطئ؟	نعم: قبل الرفع → <code>git commit --amend</code> أو <code>reset --soft</code> ؛ بعد الرفع → <code>git revert</code> . والترم كثيراً: كل التزام نقطة رجوع.
ما الفرق بين Git و GitHub بالضبط؟	Git برنامج يراقب التغييرات على جهازك. GitHub موقع يستضيف مستودعات Git ويتيح التعاون. لا يعمل أحدهما بدل الآخر.
لماذا لا أرفع مباشرة إلى <code>main</code> ؟	لأن <code>main</code> هو النسخة التي يعتمد عليها الجميع؛ الفرع يحمي الفريق حتى تُراجع المساهمة.
هل أفقد عملي إن أخطأت في الأمر؟	لا، إلا في <code>reset --hard</code> و <code>git clean -fd</code> و <code>restore</code> على تعديل غير مُلتزم. و <code>git reflog</code> يعيدك في معظم الحالات.
لماذا يظهر ملف <code>members-bundle.js</code> في كل تعديلي؟	لأنه ملف مُولّد آلياً من البطاقات. لا تعدّله يدوياً، وسيُعاد توليده مع الفهرس.
هل تعارض الدمج خطأ يجب تفاديه؟	لا، التعارض رسالة طبيعية يقول فيها Git: «هنا تعديلان متضاربان، اختر أنت». مهارة حلّه هي محور الورقة 05.
كيف أعرف ما فعله زميلي؟	<code>git log --online --graph --all</code> ثم <code>git diff</code> ثم <code>git blame</code> . وطلبات السحب على GitHub هي أوضح عرض لحالة العمل.
هل يمكن إلغاء طلب سحب؟	نعم: <code>close pull request</code> (يبقى التاريخ) أو استخدم <code>git revert</code> بعد الدمج. لا يوجد «حذف كامل» لسجل عام بل تصحيح معن.

هل يجب أن يكون مستودعي عاماً؟

كيف أستمّر بعد الورشة؟

لا في مشاريع العمل الخاصة، لكن في هذه الورشة نحتاج العام لأن GitHub Pages على حساب مجاني يعمل مع مستودعات عامة.
من أمثلة المهمة: أضف ميزة للموقع عبر فرع + طلب سحب، وشارك في مشروع مفتوح المصدر (first-contributions) لتمرين ممتع مماثل.

7 · لوحة متابعة التقدم (اطبعوها وعلّقوها) 7 · Progress board

لوحة متابعة الفريق — ثملاً أمام الطلاب / Team progress board

الاسم حساب ✓ Git مهياً بطاقة (فرع) PR فتح روجع مدموج resolve تعارض منشور المجموع / 500 · 06

قياس أثر الورشة (تقييم قبل / بعد) Measure the impact

اعط الطلاب هذا السؤال في دقيقتين قبل الجلسة الأولى وبعد الجلسة الخامسة: «اشرح بجملتين ما يحدث عندما يعدّل زميلك ملفاً ثم تدمجه، ولماذا قد يحدث تعارض.» قارن الإجابات: الفرق بين «Git يرفع الملفات» في البداية و«التزامات تجري على فرع، ثم تُدمج وتُقارن على مستوى الأسطر» في النهاية هو مقياس أثرك الحقيقي — واحتفظ بالإجابات في تقرير المؤسسة التدريبية.

8 · الملحق: خطة التقييم الكلية (500 نقطة + 100 للوحدة المتقدمة) 8 · Appendix: total assessment

الورقة	النقاط	الوزن النسبي	أهم عنصر في التقييم
01 التهيئة	100	20%	دليل اتصال مؤثّن (SSH/PAT)
02 الاستنساخ	100	20%	فهم مسار البيانات وحدث تشغيل الموقع
03 الفرع وطلب السحب	100	20%	جودة الالتزام + جودة المراجعة
04 المزامنة والدمج	100	20%	قراءة التاريخ ومختبر التراجع
05 التعارضات والنشر	100	20%	حل تعارضين + نشر الموقع
المجموع (الأساسي)	500	100%	—
06 الأتمتة (متقدم اختياري)	100	—	كسر/إصلاح موثّق + فحص أخضر + إصدار v1.0.0
المجموع الكامل مع الوحدة المتقدمة	600	—	—

توصية توزيع الدرجات النهائية (لمن يعمل بنظام التقدير الجامعي/المهني): - العملي والأدلة: 70% (بطاقة مدموجة، PR، موقع منشور، حل تعارضات) - الفهم والتوثيق: 20% (أسئلة التحقق، تقرير الفريق، جدول البنية) - الاحتراف والتعاون: 10% (جودة المراجعة، الالتزام بالاتفاقيات، البيانات والالتزام الأخلاقي)

9 · الملحق: بدائل جاهزة على GitHub 9 · Appendix: ready-made repositories

إن تعدّر عليك إنشاء منظمة ومستودع خاص بالورشة (قيود إدارية، أو شبكة مقيدة، أو عدم توفر صلاحيات)، يمكنك تنفيذ البرنامج نفسه على مستودعات تعليمية جاهزة على GitHub. الجدول يربط كل بديل بالورقة التي يخدمها.

EN

If creating your own org/repo is not possible, run the same programme on ready-made public educational repositories. The table maps each option to the worksheet it serves.

المستودع	الأفضل لـ	الفكرة	ملاحظة للمدرب
githubtraining/hellogitworld	الأوراق 04-02	مستودع تدريب رسمي من GitHub Training، يحتاج fork، والتحكم أقل يحتوي فروعاً متعددة وأخطاء مقصودة للتمرين من مستودعك	
firstcontributions/first-contributions	الورقة 03	أشهر مشروع لتمرين أول Pull Request في العالم (آلاف المساهمين)	وثق الخطوات بالعربية وأضفها لورقتك قبل الاستخدام
octocat/Spoon-Knife	الأوراق 03-02	مستودع GitHub الشهير للتمرين على fork وPR وتعديل ملف بسيط	بسيط جداً — لا يوجد كود يمكن تطويره
skills/introduction-to-github	الورقة 01	مسار تفاعلي رسمي من GitHub Skills يمنح شارات تعلم	ممتاز كواجب منزلي منفصل
(jlord/git-it-electron) git-it	الأوراق 03-01	تطبيق سطح مكتب يعلم Git بالتحديات المتدرجة	يحتاج تثبيت تطبيق إضافي
skills/resolve-merge-conflicts	الورقة 05	تمرين رسمي مخصص لحل التعارضات	استخدمه كتمرين دعم لمن تعثر في مختبر التعارض

لماذا يبقى مشروعنا هو الخيار الأول؟ Why our own project is still first choice

التدريب على مستودع جاهز يعلم **الأوامر**. أما مشروع الفريق المخصص فيعلم **العمل الجماعي المنظم**: أدواراً حقيقية، ميزانية مراجعة، ملكية ملفات، وتعاقبات على التاريخ — وهي المهارات التي يطلبها سوق العمل. لذلك: استخدم البدائل للتوسع أو التعويض، وليس كبديل أساسي. وإن لم يوجد سوى مستودع جاهز، أضف عليه خطوة إضافية: اطلب من كل طالب إضافة ملف باسمه في مجلد [students/](#) ليعيش تجربة «ملفي لا يتعارض مع ملفاتكم».

رسالة ختامية للمدرب A closing note to the trainer

نجاح هذه الورشة لا يقاس بعدد الطلاب الذين يستخدمون `git commit` بسرعة، بل بعدد الطلاب الذين يتصرفون **كزملاء في فريق** بعد الورشة: يسألون قبل أن يرفعوا، يراجعون قبل أن يمدحوا، ويقرؤون مخرجات الأوامر قبل أن يخفّنوا. صمّم تدخلك حول هذا الهدف، وستجد أن بقية المهارات التقنية تأتي تلقائياً معه.